09. Business Interpretation, Results & Limitations

Translate similarity into product value while acknowledging its limits.

Project: Movie Recommender System

Source archive supplied in this conversation: Movie-recommender-system-main.zip | Notebook: notebook.ipynb | App: app.py

Business value

- Improves discovery in a large catalogue by showing related titles from one selected movie.
- Works without a history of user ratings, so it can recommend items in a sparse-interaction setting.
- Can surface older or less popular films if their metadata overlaps meaningfully with a selected title.
- Provides a transparent reason for similarity at a token/metadata level, though the app does not expose explanations.

Main limitations

- Only metadata similarity; no personalization based on the user's tastes beyond the seed title.
- CountVectorizer uses exact word overlap and misses synonyms, context, tone and semantic relations.
- All metadata fields are concatenated without explicit weights.
- Top-three cast truncation can omit important contributors; director extraction may be empty or inconsistent.
- Merge by title may introduce duplicates/mismatches; observed vector rows (4,806) exceed original movies rows (4,803).
- No user interaction data, offline evaluation, diversity control or business KPI.
- Dense N×N similarity matrix scales poorly for large catalogues.
- Supplied ZIP omits source CSVs and generated pickle artefacts required to run the notebook/app.

App/code observations

- Streamlit app loads movie_list.pkl and similarity.pkl and requests poster images from TMDB using movie IDs.
- The API key is hard-coded in app.py. Treat it as exposed: rotate/revoke it and load secrets from environment or Streamlit secrets.
- The app uses asyncio.run() and concurrent poster requests; API errors, missing poster_path and rate limits need handling.
- The app indexes the selected movie by title and takes the first matching row; duplicate titles can return ambiguous results.
- The app expects pickle files in a model/ directory, while notebook save paths shown are current-directory paths.

Recommended next iteration

- Use movie_id as the merge key and assert expected cardinality; deduplicate titles with a deliberate policy.
- Build field-aware features or TF-IDF and tune field weights using human/user relevance labels.
- Add title disambiguation, language and genre filters, franchise/duplicate suppression and diversity re-ranking.
- Collect user feedback and build a hybrid model if interaction data is available.

- Evaluate with temporal holdouts, Precision@K, NDCG@K, coverage, diversity and popularity baselines.
- Use approximate nearest neighbours or sparse retrieval for large catalogues.
- Move API keys to secrets, handle external API errors, and align model artifact paths.

Interview conclusion

> “The prototype demonstrates a content-based retrieval approach and returns plausible neighbours for a selected movie. Its strength is that it does not require user ratings; its weakness is that it only measures metadata overlap. I would next validate recommendations with user feedback, add ranking/diversity metrics, improve feature weighting and address duplicate-title and scalability issues.”